

TrailSense Multilateration Library: Usage Guide

July 12, 2026

CONTENTS

TrailSense Multilateration Library: Usage Guide

Quickstart

Prerequisites

Step 1: Get the repo

Step 2: Configure

Step 3: Build

Step 4: Run the CLI on the sample input

What the sample carried

Next steps

How to build and link the library into your project

Path A: build in place with CMake

Path B: link into another CMake project

Path C: drop the sources in and compile directly

Path D: just use the CLI, no linking

Quick reference

How to call the library from C

A complete, minimal program

The (pointer, count) array convention

You own the output buffer

Memory ownership

Status-code handling

Caps: a direct linker must enforce the bounds itself

How to use the CLI

The round trip

From Python

Exit codes and streams

Cap warnings (exit stays 0)

When to use the CLI versus linking the library

How to handle edge cases and non-ideal results

No-position cases (Situation I Detect I Do)

Low-confidence (confidence 40) approximate bilateration fixes

Staleness: the library has no clock, track time yourself

Capacity caps: batch under the limits or lose data

How it works: the positioning model

One node gives a circle, not a point

From RSSI to range: log-distance path-loss model

Correlation: grouping sightings of the same device

The selector: how many anchors saw the device

Two anchors: bilateration and the mirror ambiguity

Three or more anchors: multilateration

Why geometry matters: diversity, collinearity, GDOP

Why surveyed anchor accuracy bounds position accuracy

What confidence and the error ellipse tell you

TrailSense Multilateration Library: Usage Guide

How to build, link, and call the library, use the CLI, handle non-ideal results, and understand the positioning model. Part of the TrailSense Multilateration Library documentation set (Usage Guide, C API Reference, Output Reference, CLI Reference, Using the Output).

Quickstart

Fresh checkout to one printed triangulated position, using the bundled sample.

Prerequisites

C11 compiler (`cc` , `gcc` , or `clang`) plus CMake 3.16+ (used below). Direct-source compilation instead: see the build-and-link how-to. The library is dependency-free C11, links only `libm` .

```
cmake --version
cc --version
```

Missing `cmake` : `brew install cmake` or `apt-get install cmake` .

Step 1: Get the repo

```
git clone <repository-url> GH-TrailSense-multilateration-library
cd GH-TrailSense-multilateration-library
```

`ls` should show `CMakeLists.txt` , `include/` , `src/` , `cli/` , `examples/` .

Step 2: Configure

```
cmake -B build
```

Creates `build/` ; compiles nothing yet.

Step 3: Build

```
cmake --build build
```

`build/` now holds:

- `libtrailsense_triangulate.a` , static C-ABI library.
- `libtrailsense_triangulate.so` (`.dylib` on macOS), shared C-ABI library.
- `trailsense-triangulate` , the CLI.

Step 4: Run the CLI on the sample input

```
./build/trailsense-triangulate < examples/sample-input.json
```

Reads the JSON batch on stdin, writes JSON on stdout (one line; pretty-printed here):

```
{
  "positions": [
    {
      "mac_hash": "deadbeef",
      "band": 0,
      "signal_type": "wifi",
      "fingerprint_hash": "w_deadbeef",
      "lat": 44.999675310,
      "lon": -93.000735291,
      "accuracy_m": 17.522416693,
      "measurement_count": 3,
      "confidence": 70,
      "peak_rssi": -55,
      "error_semi_major_m": 17.522416693,
      "error_semi_minor_m": 17.522416693,
      "error_major_axis_deg": 0.000000000
    },
    {
      "mac_hash": "cafef00d",
      "band": 1,
      "signal_type": "bluetooth",
      "fingerprint_hash": "b_cafef00d",
      "lat": 45.000000000,
      "lon": -93.000000000,
      "accuracy_m": 984.562871623,
      "measurement_count": 2,
      "confidence": 40,
      "peak_rssi": -70,
      "error_semi_major_m": 984.562871623,
      "error_semi_minor_m": 79.430000000,
      "error_major_axis_deg": 0.000000000
    }
  ]
}
```

Two positions means the library works end to end.

What the sample carried

Input = surveyed anchor positions (fixed `lat` / `lon` per node) plus raw observations (each node's RSSI sighting of a device, keyed by `mac_hash` and `band`). The library correlates observations by device, converts RSSI to range via the path-loss model, and solves each device's location. Distinct-anchor

count selects the solver and confidence: two anchors give an approximate bilateration fix (`cafef00d`, `confidence` 40, large uncertainty); three or more give a full triangulated fix (`deadbeef`, `confidence` 70).

Next steps

- Integrate (link, vendor, or subprocess): the Build and link section.
- Full schema, C API, status codes, anchor-count rule: the C API Reference and Output Reference.
- Verify bit-exactness: `ctest --test-dir build` (parity suite + CLI golden test).

How to build and link the library into your project

Pick the integration path that fits your build and wire the library in.

Two facts every path assumes: link only `libm` (`-lm` on direct compile/link); the single public header is `include/trailsense_triangulate.h`, so the include path needs `include/`.

Compile the core with `-fno-fast-math`: the math needs standard IEEE-754 `double` semantics for bit-exact parity with the reference backend. The bundled `CMakeLists.txt` enforces it; compiling yourself, pass `-fno-fast-math` and never `-ffast-math`.

Path A: build in place with CMake

```
cmake -B build
cmake --build build
```

In `build/`:

- `libtrailsense_triangulate.a`, static C-ABI library.
- `libtrailsense_triangulate.so` (`.dylib` on macOS), shared C-ABI library.
- `trailsense-triangulate`, the CLI.

Verify:

```
ctest --test-dir build
```

Runs the parity suite (math vs reference fixtures) and `cli_golden` (built CLI on `examples/sample-input.json`, diffed against `examples/sample-output.json`). All passing proves the build is bit-exact and the JSON plumbing stable.

Path B: link into another CMake project

Link target `trailsense_triangulate` (static library); it exports the `include/` path and the `libm` link, so add neither yourself.

B1, vendored subdirectory (for example under `third_party/`):

```
add_subdirectory(third_party/GH-TrailSense-multilateration-library)

add_executable(your_app your_app.c)
target_link_libraries(your_app PRIVATE trailsense_triangulate)
```

B2, FetchContent (fetched at configure time):

```
include(FetchContent)
FetchContent_Declare(
  trailsense_triangulate
  GIT_REPOSITORY <repository-url>
  GIT_TAG        <tag-or-commit>
)
FetchContent_MakeAvailable(trailsense_triangulate)

add_executable(your_app your_app.c)
target_link_libraries(your_app PRIVATE trailsense_triangulate)
```

In your source:

```
#include "trailsense_triangulate.h"
```

Path C: drop the sources in and compile directly

No CMake. Compile these six core files from `src/` alongside your own:

- `src/pathloss.c`
- `src/enu.c`
- `src/multilaterate.c`
- `src/correlate.c`
- `src/solve.c`
- `src/triangulate.c`

Add `include/` with `-Iinclude`, compile as C11, link `libm` with `-lm`, pass `-fno-fast-math`:

```
cc -std=c11 -fno-fast-math -Iinclude \
  your_app.c \
  src/pathloss.c src/enu.c src/multilaterate.c \
  src/correlate.c src/solve.c src/triangulate.c \
  -lm -o your_app
```

Do not compile `cli/main.c` into the core: it is the CLI front end (own `main`), not part of the library. Include the header:

```
#include "trailsense_triangulate.h"
```

Path D: just use the CLI, no linking

Non-C host or no native linking. Build once, call as a subprocess: input JSON to stdin, result JSON from stdout.

```
cmake -B build
cmake --build build
```

```
./build/trailsense-triangulate < your-input.json > result.json
```

Reads one JSON object `{ "anchors": [...], "obs": [...] }` on stdin, writes `{ "positions": [...] }` on stdout. Fails loud: descriptive message on stderr, nonzero exit; check the exit code. Over-cap input warns on stderr but still exits 0 with the partial result. Schema in the CLI Reference; `examples/sample-input.json` is a working start.

Quick reference

- Include path: `include/`
- Public header: `include/trailsense_triangulate.h`
- C standard: C11
- Link dependency: `libm` (`-lm`), and nothing else
- Required compile flag for parity: `-fno-fast-math` (never `-ffast-math`)
- Core source files: `src/pathloss.c`, `src/enu.c`, `src/multilaterate.c`, `src/correlate.c`, `src/solve.c`, `src/triangulate.c`
- CMake link target: `trailsense_triangulate`
- Build verification: `ctest --test-dir build` (parity + `cli_golden`)

How to call the library from C

Link `libtrailsense_triangulate`, fill your own observation and anchor arrays, read positions from a buffer you own.

A complete, minimal program

Locates one Wi-Fi device (`deadbeef`) seen by three anchors; values mirror `examples/sample-input.json`.

```

#include <stdio.h>
#include <stddef.h>
#include "trailsense_triangulate.h"

int main(void) {
    /* Three surveyed anchors. enrolled = 1 keeps them; path-loss overrides
     * left at 0 mean "use the band default" for that field. */
    ts_anchor_t anchors[] = {
        { .node_id = 1, .lat = 45.0,      .lon = -93.0,      .enrolled = 1 },
        { .node_id = 2, .lat = 45.00009, .lon = -93.0,      .enrolled = 1 },
        { .node_id = 3, .lat = 45.000045, .lon = -92.99991, .enrolled = 1,
          .path_loss_a_cdb = 4000, .path_loss_n_milli = 2000 },
    };
    size_t n_anchors = sizeof anchors / sizeof anchors[0];

    /* Raw observations: three nodes each saw device deadbeef on band 0 (Wi-Fi). */
    ts_obs_t obs[] = {
        { .node_id = 1, .mac_hash = {0xde,0xad,0xbe,0xef}, .band = TS_BAND_WIFI,
          .rssi = -55, .ts_ms = 1000 },
        { .node_id = 2, .mac_hash = {0xde,0xad,0xbe,0xef}, .band = TS_BAND_WIFI,
          .rssi = -60, .ts_ms = 1050 },
        { .node_id = 3, .mac_hash = {0xde,0xad,0xbe,0xef}, .band = TS_BAND_WIFI,
          .rssi = -62, .ts_ms = 1100 },
    };
    size_t n_obs = sizeof obs / sizeof obs[0];

    /* You own the output buffer. Size it to how many distinct (mac_hash, band)
     * groups you expect; TS_MAX_GROUPS (64) is the library's hard ceiling. */
    ts_position_t out[TS_MAX_GROUPS];
    size_t out_n = 0;

    int rc = ts_triangulate(obs, n_obs, anchors, n_anchors,
                           out, sizeof out / sizeof out[0], &out_n);
    if (rc != TS_OK) {
        fprintf(stderr, "ts_triangulate failed: %d\n", rc);
        return 1;
    }

    printf("%zu position(s)\n", out_n);
    for (size_t i = 0; i < out_n; i++) {
        const ts_position_t *p = &out[i];
        printf("mac=%02x%02x%02x%02x band=%d lat=%.9f lon=%.9f "
              "accuracy_m=%.3f count=%d confidence=%d peak_rssi=%d\n",
              p->mac_hash[0], p->mac_hash[1], p->mac_hash[2], p->mac_hash[3],
              p->band, p->lat, p->lon, p->accuracy_m,
              p->measurement_count, p->confidence, p->peak_rssi);
    }
    return 0;
}

```

Build and run (adjust paths to your build tree):

```
cc example.c -I include -L build -ltrailsense_triangulate -lm -o example
./example
```

Expected: one position, `count=3`, `confidence=70` (three anchors give full multilateration).

The (pointer, count) array convention

Every array crossing the ABI is a pointer plus a separate `size_t` count: `(obs, n_obs)` and `(anchors, n_anchors)` in, `(out, out_cap)` plus written-count `out_n` out. No NUL- or sentinel-terminated arrays; pass the exact count and the library reads exactly that many.

You own the output buffer

Allocate `out` (stack, `.bss`, or heap), pass capacity as `out_cap`. The library writes at most `out_cap` positions, sets `*out_n` to the count. Read only `out[0 .. *out_n - 1]`. Positions exceeding `out_cap` return `TS_ERR_OUT_T00_SMALL` with no usable partial result. One position per distinct resolving `(mac_hash, band)` group, so `out` of `TS_MAX_GROUPS` (64) elements always holds every position one call can emit; a smaller buffer is fine only when the batch has fewer groups.

Memory ownership

Allocates nothing on the heap, keeps no global state; every call is reentrant. Borrows `obs` and `anchors` read-only for the call (nothing retained after return), writes into `out`, and holds no reference to your buffers after return. Free or reuse immediately.

Status-code handling

`ts_triangulate` returns an `int`:

Code	Value	Meaning
<code>TS_OK</code>	<code>0</code>	success; <code>*out_n</code> positions written (including 0)
<code>TS_ERR_NULL_ARG</code>	<code>-1</code>	a required pointer argument was NULL
<code>TS_ERR_OUT_T00_SMALL</code>	<code>-2</code>	<code>out_cap</code> too small for the positions produced
<code>TS_ERR_BAD_BAND</code>	<code>-3</code>	an observation carried a band outside <code>0..2</code>

`TS_OK` with `*out_n == 0` is normal, not an error: no group had two or more distinct enrolled, resolvable anchors. Branch on `rc != TS_OK` first, then loop over `*out_n`; treat any negative return as a hard failure and do not read `out`.

Caps: a direct linker must enforce the bounds itself

One call processes at most `TS_MAX_GROUPS` (64) distinct `(mac_hash, band)` groups and `TS_MAX_NODES_PER_GROUP` (32) distinct nodes per group. Input beyond either bound is **silently dropped**, yielding a partial result that still returns `TS_OK`; the core never prints or signals truncation. Linking directly you get no warning, so keep each batch under both caps: if a batch could exceed 64 groups or 32 nodes for one device, split it and call `ts_triangulate` once per batch, or lose data with no error. (The bundled CLI adds a stderr warning for over-cap input; the library does not.)

How to use the CLI

Shell out to `trailsense-triangulate` from any language: JSON batch on stdin, JSON result on stdout.

The round trip

```
cat examples/sample-input.json | build/trailsense-triangulate
```

Input (`examples/sample-input.json` ; fields explained in the Quickstart section, `mac_hash` = 8 hex chars / 4 bytes):

```
{
  "anchors": [
    {"node_id": 1, "lat": 45.0,      "lon": -93.0,      "enrolled": true},
    {"node_id": 2, "lat": 45.00009,  "lon": -93.0,      "enrolled": true},
    {"node_id": 3, "lat": 45.000045, "lon": -92.99991, "enrolled": true,
     "path_loss_a_cdb": 4000, "path_loss_n_milli": 2000}
  ],
  "obs": [
    {"node_id": 1, "mac_hash": "deadbeef", "band": 0, "rssi": -55, "ts_ms": 1000},
    {"node_id": 2, "mac_hash": "deadbeef", "band": 0, "rssi": -60, "ts_ms": 1050},
    {"node_id": 3, "mac_hash": "deadbeef", "band": 0, "rssi": -62, "ts_ms": 1100},
    {"node_id": 1, "mac_hash": "cafef00d", "band": 1, "rssi": -70, "ts_ms": 1200},
    {"node_id": 2, "mac_hash": "cafef00d", "band": 1, "rssi": -72, "ts_ms": 1250}
  ]
}
```

Output (one line on stdout):

```
{"positions":
[{"mac_hash":"deadbeef","band":0,"signal_type":"wifi","fingerprint_hash":"w_deadbeef","lat"
{"mac_hash":"cafef00d","band":1,"signal_type":"bluetooth","fingerprint_hash":"b_cafef00d","
```

`deadbeef` (three anchors) triangulates, `confidence` 70; `cafef00d` (two anchors) gives an approximate bilateration fix, `confidence` 40.

From Python

Subprocess: write request JSON to stdin, read response JSON from stdout, parse.


```

import json
import subprocess

request = {
    "anchors": [
        {"node_id": 1, "lat": 45.0, "lon": -93.0, "enrolled": True},
        {"node_id": 2, "lat": 45.00009, "lon": -93.0, "enrolled": True},
        {"node_id": 3, "lat": 45.000045, "lon": -92.99991, "enrolled": True},
    ],
    "obs": [
        {"node_id": 1, "mac_hash": "deadbeef", "band": 0, "rssi": -55, "ts_ms": 1000},
        {"node_id": 2, "mac_hash": "deadbeef", "band": 0, "rssi": -60, "ts_ms": 1050},
        {"node_id": 3, "mac_hash": "deadbeef", "band": 0, "rssi": -62, "ts_ms": 1100},
    ],
}

proc = subprocess.run(
    ["build/trailsense-triangulate"],
    input=json.dumps(request),
    capture_output=True,
    text=True,
)

if proc.returncode != 0:
    # Bad input. The diagnostic is on stderr.
    raise RuntimeError(f"triangulate failed: {proc.stderr.strip()}")

if proc.stderr:
    # Exit 0 with stderr text means a cap warning; the result is partial.
    print("warning:", proc.stderr.strip())

result = json.loads(proc.stdout)
for pos in result["positions"]:
    print(pos["fingerprint_hash"], pos["lat"], pos["lon"], pos["confidence"])

```

Same pattern from Node.js (`child_process.execFile` / `spawn`), Go (`os/exec`), Rust, or any runtime that spawns a process and pipes stdin/stdout: JSON in, JSON out, check the exit code.

Exit codes and streams

- **stdout carries data only:** on success, exactly one JSON object `{"positions": [...]}`. Parse stdout, never stderr.
- **stderr carries diagnostics only:** error messages and cap warnings, never JSON.
- **exit 0 = success**, including zero positions: no resolvable group still exits 0 and prints `{"positions": []}`.
- **nonzero exit = bad input** (missing required field, `mac_hash` not 8 hex chars, out-of-range `band`), with a descriptive stderr message. No position emitted.

Cap warnings (exit stays 0)

The core silently drops input past its caps (`TS_MAX_GROUPS` = 64 distinct `(mac_hash, band)` groups; `TS_MAX_NODES_PER_GROUP` = 32 distinct nodes per group). The CLI warns on stderr but **still exits 0 and emits the partial result**, so nonempty stderr alone is not failure: check the exit code to tell an error (nonzero) from a cap warning (zero). Need every group and node? Keep each batch under both caps.

When to use the CLI versus linking the library

Prefer the CLI when: host is not C and you want a clean process boundary (Python, Node, Go, etc.); no C toolchain in the deployment environment; or you want process isolation so a bad batch cannot corrupt your host process.

Prefer linking the library (see the C how-to) when: triangulation is on a hot path and per-call process-spawn overhead matters; you want the result in-process with no JSON round trip; or you already write C/C++ and can own the input and output buffers directly.

How to handle edge cases and non-ideal results

Detect and respond when the library emits no position, a low-confidence position, or silently drops input.

The library is conservative: too-weak geometry emits nothing. “No output for a device” is a normal, expected signal, not an error. In every case here `ts_triangulate` still returns `TS_OK`; detect by inspecting output, not a status code. No-position devices are simply absent from output (no marker, no error code) - detect by comparing observed `mac_hash` values against those in `out[0 .. *out_n - 1]` (or the CLI `positions` array); any observed device missing from output failed to produce a fix.

No-position cases (Situation I Detect I Do)

Situation	Cause / detect	What to do
0 or 1 resolvable anchors	Device seen by fewer than 2 distinct enrolled, resolvable anchors. “Resolvable” = observing node’s <code>node_id</code> has an anchor entry with <code>enrolled</code> set. Absent from output.	Confirm observing nodes have enrolled anchor entries (common cause: node missing from table or <code>enrolled</code> =0); fix the table. One node’s RSSI gives only a range (a circle), never a point, so single-node coverage cannot locate a device. Coverage/deployment fact, not a software bug.
Degenerate/collinear geometry (3+ path only)	Geometric-diversity gate rejects sets whose bounding-box diagonal spans less than about 10 m, OR whose minor-axis spread is less than 30% of the major-axis spread (near-collinear). Absent despite 3+ resolvable anchors (or the outlier path failed to keep 3 survivors).	Placement problem, prevent at survey time: spread anchors in 2D as a triangle with meaningful area, not a line; avoid tight clusters (within a ~10 m box); aim for a compact, well-spread triangle or better around the coverage zone.
Near-coincident 2-node pair	2-node group whose anchors are closer than the minimum baseline (<code>TS_MIN_BILATERATION_BASELINE_M</code> , default 5 m) is dropped. Along-baseline uncertainty is unbounded as baseline nears 0, so a pair a few meters apart (one sensor surveyed twice with GPS jitter, two units on one pole) would else emit a confident multi-kilometer ellipse. Absent despite exactly 2 resolvable anchors.	Separate the two anchors by well more than 5 m. Check for duplicate/jittered survey coordinates (a duplicate <code>node_id</code> in the anchor table resolves last-wins). Tune per deployment via <code>TS_MIN_BILATERATION_BASELINE_M</code> in <code>src/internal.h</code> ; lowering admits shorter baselines at the cost of larger, less trustworthy ellipses.

Low-confidence (confidence 40) approximate bilateration fixes

A device seen by exactly 2 resolvable anchors produces an approximate bilateration fix with `confidence == 40` (equivalently `measurement_count == 2`). With only two range circles the position is unresolvable off the anchor baseline, so the library reports the range-consistent point on the baseline.

- Treat as approximate: useful for coarse location, not a precise fix.
- Read the ellipse correctly (mixed semantics). The along-baseline axis (`error_semi_major_m` when it dominates) is a roughly 1-sigma range uncertainty. The cross-baseline axis is a deterministic mirror-ambiguity bound, NOT a confidence radius: do not draw it as a Gaussian error circle and do not feed it to a filter expecting a 1-sigma value. See the Output Reference.
- Expect a large along-baseline axis when the baseline is short relative to target range. Honest uncertainty (the `r / d` term), not an error.
- To upgrade: add a 3rd resolvable anchor. Anchor count is the only selector, so a 3rd enrolled unit that sees the device moves the group to confidence 70 with no configuration change.

Staleness: the library has no clock, track time yourself

The library is stateless and has no clock. It uses observation `ts_ms` only for latest-per-node selection and age-weighting within a single batch; it never rejects a batch for age and never stamps output with a time. There is no timestamp field on output, so you cannot detect staleness from library output. Track time outside the library:

- Record each batch's receive/observation time and associate it with the returned positions.
- Reject or age-out stale fixes in your own layer: pick a maximum acceptable age and discard older positions.
- Bound each batch by time before calling (the caller owns windowing): pass e.g. only the last 10 seconds per call. Mixing very old and very recent observations in one batch produces a blended fix.

Capacity caps: batch under the limits or lose data

One `ts_triangulate` call processes at most `TS_MAX_GROUPS` (64) distinct `(mac_hash, band)` groups and at most `TS_MAX_NODES_PER_GROUP` (32) distinct nodes per group. Input beyond either bound is silently dropped by the core, yielding a partial result that still returns `TS_OK`; the core does not warn. (The bundled CLI warns on stderr on over-cap input and still exits 0 with the partial result; a direct linker gets no warning.)

- Count distinct groups and per-group nodes before calling; exceeding 64 groups or 32 nodes/device loses data silently.
- Split oversized batches so each call stays under both caps, one call per partition. Splitting by group is safe: each group solves independently.

- Size the output buffer to `TS_MAX_GROUPS`. A 64-element `out` always holds every position one call can emit, so you never hit `TS_ERR_OUT_T00_SMALL`. See the Call from C section and the C API Reference.

How it works: the positioning model

How the library turns RSSI from several sensor nodes into a position, and why accuracy, confidence, and the error ellipse look as they do. The C API Reference and Output Reference are authoritative.

One node gives a circle, not a point

A node measures signal strength (RSSI), not direction. Signal strength falls off with distance, so RSSI yields an estimated distance (a range), not a direction. A range describes a circle: the device is anywhere on that ring, so one node locates only to a circle. A second node gives a second circle; two circles generally intersect at two points (an ambiguity, the mirror problem below). A third circle picks one of those two, and three or more well-placed nodes pin the device to a single point. Turning several ranges into one position is the library's whole job.

From RSSI to range: log-distance path-loss model

Received power decreases with the logarithm of distance: doubling distance costs a roughly fixed number of dB, and dB-per-doubling depends on the environment. Two parameters govern the conversion:

- Reference term `A`: expected path loss at a reference distance, anchoring the curve.
- Path-loss exponent `n`: how fast the signal fades. Free space is near 2; cluttered outdoor environments push it higher.

The device's assumed transmit power completes it: range is derived from how much the signal weakened between device and node, given `A`, `n`, and transmit power. Range is exponential in RSSI through this model, so a wrong `n` or `A` bends every range estimate, and a systematic range bias biases the final position. The library ships OUTDOOR band-prior defaults for Wi-Fi, BLE, and cellular, and lets you override `A` and `n` per anchor when you have calibrated an environment. Good path-loss parameters matter as much as good anchor coordinates; if ranges are systematically wrong, no geometry recovers the true position. Ranges are clamped to a sane band (roughly half a meter to a few hundred meters) so a freak RSSI cannot produce an absurd range.

Correlation: grouping sightings of the same device

The library groups observations by the pair `(mac_hash, band)` (device fingerprint hash and radio band). Nodes that saw the same fingerprint on the same band form one group, solved independently into at most one position. A node may report a device more than once per group; the library keeps the latest sample per node by timestamp, so each node contributes one range. Timestamps also age-weight samples (fresher counts for more) but never split the batch into time windows - batch time span is the caller's job.

The selector: how many anchors saw the device

The next step depends entirely on one number: how many distinct enrolled, resolvable anchors are in the group (“resolvable” = observing node has a surveyed, enrolled anchor entry). That count, and only it, selects the method:

- 0 or 1 anchor: no position (a single circle cannot locate a point).
- Exactly 2 anchors: bilateration, the approximate two-node method, lowest confidence tier.
- 3 or more anchors: multilateration, the full method, higher confidence tier.

Because the count is the only switch, a deployment improves itself: add a third sensor covering an area and every device seen there moves from the approximate method to the full one, with no reconfiguration.

TWO ANCHORS: BILATERATION AND THE MIRROR AMBIGUITY

Two circles generally cross at two points, one on each side of the line joining the anchors (the baseline). The two nodes alone cannot tell them apart; the device is equally consistent with either. This is the mirror ambiguity. The library reports the point on the baseline consistent with both ranges (the foot of the perpendicular between the two intersection points), and describes remaining uncertainty with an ellipse whose axes mean two different things:

- Along the baseline: a genuine statistical spread. Noisy ranges slide the answer along the baseline. Behaves like a roughly one-sigma error and grows when the baseline is short relative to target distance.
- Across the baseline: not statistical. It is the mirror ambiguity: the true device sits a fixed distance off the baseline (one side or the other) while the reported point sits on it. A hard bound on how far off the baseline the device can be, not a bell-curve spread.

Hence the two-node ellipse has mixed semantics, and a consumer must not treat the across-baseline axis as a confidence radius. It says “the device is within this far off the baseline, on one side or the other,” not “68 percent chance the device is within this radius.”

THREE OR MORE ANCHORS: MULTILATERATION

With three or more circles the mirror ambiguity resolves: the extra circle selects which side of any baseline the device is on, and the intersection converges to a single point. The library solves for the position best fitting all ranges at once, using a robust iterative method that down-weights or discards measurements that disagree badly (a wall reflection, a momentary fade); robustness matters because RF ranges are noisy and one bad reading should not drag the fix. Reported accuracy here is residual-based: how well, on average, the final position agrees with individual ranges. The error ellipse is an isotropic placeholder (a circle of radius = accuracy), because full directional covariance is not computed for three-plus-node fixes. The position and its scalar accuracy are the trustworthy outputs; read no directional meaning into the circle.

Why geometry matters: diversity, collinearity, GDOP

Enough anchors is necessary but not sufficient. Where anchors sit relative to each other and the device determines how much the ranges constrain the answer, this is geometric dilution of precision (GDOP): the same range noise gives a small position error under good geometry, a large one under poor geometry. The worst geometry is collinear anchors: three nodes on a near-straight line constrain the device in essentially one direction and barely in the perpendicular, so small range error smears the position badly along the weak axis. Tightly packed anchors do the same (a small footprint gives little leverage).

The three-plus path guards against this with a geometric-diversity gate, rejecting sets whose footprint is too small (bounding box not spanning a meaningful distance) or too close to a straight line (one spread axis far smaller than the other). A rejected set yields no position, because a position from degenerate geometry would be confidently wrong. Survey anchors to surround the coverage area in a well-spread triangle or better, not a line or cluster. The two-node path skips this gate, since two points are always collinear and bilateration already accounts for it.

Why surveyed anchor accuracy bounds position accuracy

Every position is computed relative to the anchor coordinates you supply, so the math is only as accurate as those coordinates. An anchor surveyed 20 meters off makes every fix using it inherit that offset. Parity with the reference backend proves the arithmetic is correct; it says nothing about whether anchors are where you claim. The library cannot detect a badly surveyed anchor, because a wrong-but-plausible coordinate produces a wrong-but-plausible position. Careful sensor surveying is a precondition for meaningful output, not an optional refinement. Placeholder or synthetic anchor coordinates give output that is internally consistent and geographically meaningless.

What confidence and the error ellipse tell you

Two output fields describe trust and answer different questions. Confidence is a coarse tier reflecting the method used, which reflects anchor count: the approximate two-node tier, the three-anchor tier, and the four-or-more tier. It is not a probability; read it as “how much geometry was behind this fix,” where more anchors mean a more constrained, more trustworthy result. It does not measure the actual error of a specific fix. The error ellipse describes the shape and size of uncertainty for a specific fix: on the two-node path it carries real geometric meaning in two flavors (along-baseline statistical spread, across-baseline mirror bound); on the three-plus path it collapses to a circle sized by the residual accuracy. Confidence tells you which regime you are in; the ellipse tells you how tight the answer is within that regime.